

Music : Phonetic factors and popularity

Shiwam Birajdar

25BM6JP07

1 INTRODUCTION

1.1 Objective

The objective of this project is to identify the phonetic factors that impact the popularity of a song, and build a classification model that is robust at identifying a popular song, based on the phonetic factors available in the public Spotify data set[4]. The classification model that the project aims to use is an **XGBClassifier model**.

1.2 Problem Definition

The problem at hand is a binary classification task. It is defined on the basis of the parameters available through the Spotify 114k[4] data set, which provides information on the phonetic factors and the popularity scores.

Input: The values for the phonetic factors : 'explicit','danceability','energy','key','loudness','mode','speechiness','acousticness','instrumentalness','liveness','valence','tempo','time signature'(As defined in the Figure1)

Output: Class '1' or Class '0'

Where 1 : higher chances of being popular, 0 : lower chances of being popular.

The classes '1' and '0' are based on the popularity score, where a song with a popularity score higher than 30 is considered popular, and not popular otherwise. This threshold was set after a careful consideration, and with the reference of Groover[3] - a site used by upcoming artists to judge their popularity score. To chose the best model, and evaluate, AUC-ROC scores and Recall scores for class '1' are considered, as they align with our objective of building a model that is able to predict or identify a popular song, based solely on its phonetic factors.

1.3 Contributions

The state of the art methods for identifying the popularity of a song involve the use of CNNs, with the support of the Audio waveform spectrograms[1][2].The studies also compare the performances of other models such as Logistic Regression, Random Forest Classifiers and Support Vector Machines. This project focuses on using the XGBoost classifier model, an ensemble method, to asses the same objective. This is an attempt to build a more accessible model to up and coming artists with minimal access to resources, yet robust enough to achieve the goal. It also sets a benchmark value for the use of an XGBoost classifier model in the song popularity prediction space. The XGBoost classifier model is a supervised learning, and an ensemble method, which the project leverages, to tune several hyper parameters that contribute to a better estimate, and focuses on Ridge Regularization to prevent overfitting. In short, the project contributes to the space by providing an XGBoost based method, which is robust, adaptable, and works with minimal resources to achieve the said goal.

Phonetic factor	Description	Phonetic factor	Description
Acousticness	Confidence the track is acoustic (0-1).	Loudness	Loudness in decibels (dB).
Danceability	Suitability for dancing (0-1).	Mode	Modality (major-1, minor-0).
Energy	Intensity and activity measure (0-1).	Speechiness	Presence of spoken words (0-1).
Instrumentalness	Likelihood of no vocals (0-1).	Tempo	Tempo in BPM.
Key	Key of the track (0-11).	Time Signature	Beats per bar.
Liveness	Presence of an audience (0-1).	Valence	Musical positiveness (0-1).
Explicit	Explicitness of lyrics 0 or 1		

Figure 1. Definitions of parameters

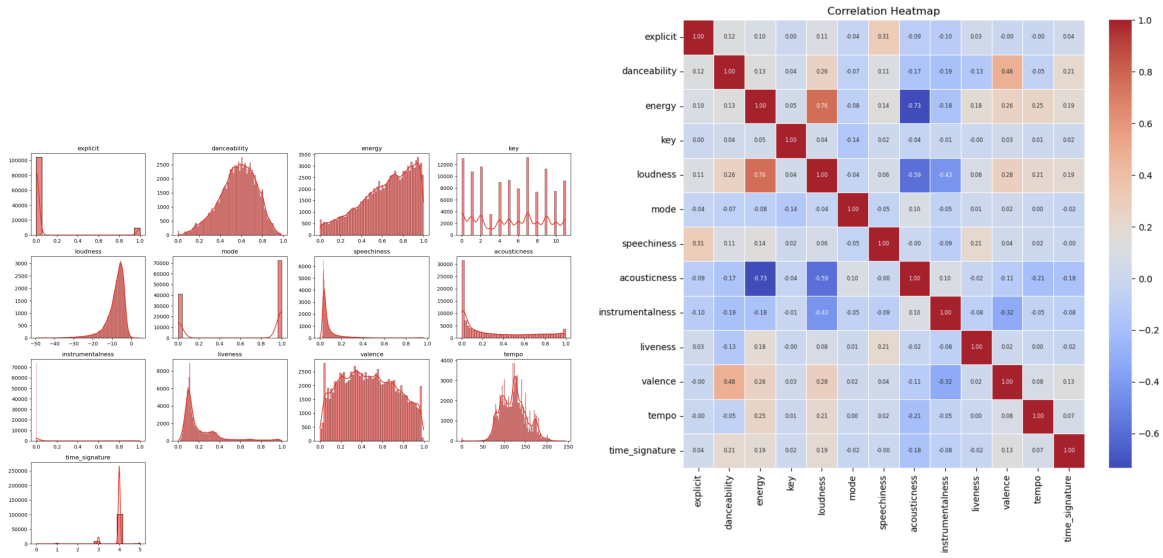


Figure 2. (a)Distributions of the features, (b)Correlation Heatmap of the features

2 METHODOLOGY

2.1 Data Pre-processing and Exploratory Analysis

The study began with a selection of thirteen of the phonetic features, including "danceability," "energy," and "explicit" features. An important initial step involved generating a correlation heatmap to analyze the interdependencies among these features. The visualization, Figure 5(b), revealed a few significant correlations between energy and loudness, energy and acousticness, acousticness and loudness, instrumentalness and loudness, and valence and danceability, indicating a very high chance of multicollinearity. This indicated that the use of regularization was necessary. Ridge Regularization was chosen as it offers variable shrinkage but does not diminish them, simultaneously. The data was subsequently split into training and testing sets in an 80:20 ratio, also using stratification to maintain the class distribution observed in the original dataset.

The popularity scores range from 0 to 100. And, as the information obtained from Groover[3], a site for up and coming artists, it would be apt to use the score 30 as a threshold for the popular and not so popular songs, above 30 being the former. The data set was then labeled with classes '1' for popular and '0' for not popular, based on their scores.

2.2 Model Architecture and Validation Strategy

To build a robust classification system, a scikit-learn Pipeline that integrates StandardScaler with an XGBoost Classifier was chosen. This architecture ensures that feature scaling is applied within each fold of the cross-validation process, as it is evident from Figure5(a) that the ranges of each factor vary significantly, and maintaining a scaled structure while training and validating every fold is necessary. Stratified K-Fold Cross-Validation of k=10 splits was used as it is a standard amount to achieve the balance between bias and variance for model evaluation. This approach is particularly effective for this data set, as it is slightly imbalanced (63513:50487) when it comes to the classes '1' and '0'.

2.3 Hyperparameter Optimization and Overfitting Control

The final phase focused on rigorous hyperparameter tuning using RandomizedSearchCV with 200 iterations, optimizing specifically for the ROC AUC score, as the project's focus is on being able to distinguish between the classes extremely well. The search space was designed with ranges relevant to the dataset, such as bounding the max_depth between 3 to 5, and keeping the estimators below 900, as the number of attributes is thirteen, and using scale_pos_weight to deal with class imbalance, to prevent overfitting and ensure proper unbiased training of the data. Specifically, **n_estimators**, **learning_rate**, **max_depth**, **min_child_weight**, **gamma**, **subsample**, **col_samplebytree**, **reg_lambda**, **max_delta_step**, and **scale_pos_weight** parameters were considered for tuning. Apart from adjusting their ranges according to the nature of the data set, the model AUC-ROC scores were observed iteratively to narrow

Hyperparameter	Description	Best Value	Reason to include
subsample	Proportion of training data sampled	0.8	Reduce Overfitting
scale_pos_weight	Weightage given to class '1'	0.9	Handle imbalance
reg_lambda	Ridge regularization	3	Regularization
n_estimators	Number of estimators	700	Reduce Overfitting
min_child_weight	minimum sum of Hessians required in a child node for further splitting to occur	1.2	Regularization
max_depth	Maximum depth of a tree	5	Reduce Overfitting
max_delta_step	limits the maximum delta allowed in the output prediction of each tree's leaf node	2.5	Reduce Overfitting
learning_rate	learning rate to find optima	0.19	Reduce Overfitting
gamma	minimum loss reduction required to make a further split on a leaf node	0	Regularization
colsample_bytree	Proportion of columns sampled by tree	0.9	Reduce Overfitting

Figure 3. Hyper parameter values for the best model

--- Final Test Set Evaluation ---		Classification Report:			
ROC AUC Score: 0.7869		precision recall f1-score support			
PR AUC Score (Precision-Recall): 0.8021	0	0.71	0.63	0.67	10097
	1	0.73	0.79	0.76	12703
Confusion Matrix:		accuracy			
[[6366 3731]		macro avg			
[2606 10097]]		weighted avg			
		0.72	0.71	0.71	22800
		0.72	0.72	0.72	22800

Figure 4. Results from the Test (unseen) Data

the search ranges of the other hyper parameters. This comprehensive search allowed the model to find a configuration that maximizes discriminative power while maintaining strict control over variance, validated finally against the unseen test set. Alpha regularization i.e. Lasso was also considered but it had no effect on the scores and was subsequently omitted from the later iterations.

3 RESULTS

The model is able to achieve our objective with a pretty good score

3.1 Quantitative Results

After several iterations of the 2000 fits i.e. 200 combinations of 10 fold fits, the best model(Figure 3) was selected based on the best ROC-AUC score and a subsequent good Recall score for class '1' as that aligns with our objective.

The selected model was then tested on unseen data, with the results displayed in Figure 4. The model has a recall value 0.79 for Class 1, and ROC-AUC score of 0.7869. It can also be noted that good or above average values, across the classes and metrics, are observed. The main focus of the project is to be able to predict a popular song, and the metrics that judge Class 1 and the overall discrimination power (Recall,ROC-AUC) are very good across the Test set. It can also be noted that other metrics such as precision, recall and others for Class 0, are also good or above average. Hence, maintaining an appropriate balance.

3.2 Qualitative Results

The model is able to strongly predict if the song will be popular based on the given phonetic factors. After an analysis of the importance of features in the model (Figure 5) we can say that the model predicts higher chances of popularity for songs that are moody, vocal-driven, and danceable, rather than high-energy or purely happy tracks. Valence is the most critical feature, showing a negative correlation i.e. low valence (blue dots, representing sadder or more serious tones) strongly drives predicted popularity up, while high valence (happy) pushes it down. The model also favors low energy and low instrumentality, suggesting a preference for "chill" vocal-heavy tracks over aggressive or purely instrumental ones. However, danceability and explicit content have a positive impact—higher values in these features which boost the chances of popularity. Conversely, high speechiness and liveness (likely indicating live recordings or spoken word) affect negatively, dragging the predicted chances of popularity down substantially. These results can help an up and coming artist to take appropriate measures to compose a popular song.

4 CODE EXECUTION INSTRUCTIONS

These are the necessary steps, environment requirements, and file dependencies required to produce the test set evaluation results.

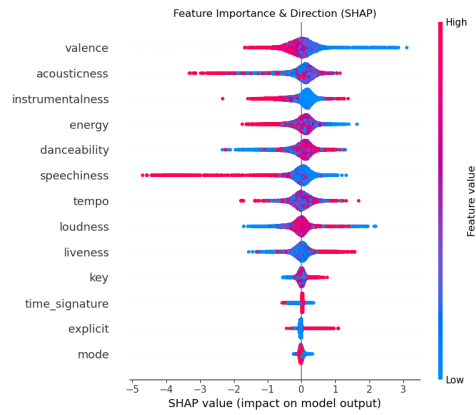


Figure 5. Feature Importance

4.0.1 System Requirements

The code requires a Python 3 environment (Python 3.8 or higher is recommended). The following Python libraries must be installed to execute the script successfully.

- `pandas`: For data manipulation and loading CSV files.
- `numpy`: For numerical operations.
- `scikit-learn`: For metrics (ROC AUC, Confusion Matrix) and pipeline handling.
- `xgboost`: Required as the backend for the loaded model pipeline.
- `joblib`: For loading the serialized model pipeline.

You can install these dependencies using `pip`:

```
pip install pandas numpy scikit-learn xgboost joblib
```

4.0.2 File Directory Structure

Ensure that the Python script is placed in a directory containing the following three essential files. The script relies on relative paths and will fail if these files are missing:

- `best_xgb_pipeline.pkl`: The pre-trained XGBoost pipeline object.
- `X_test.csv`: The test dataset features.
- `y_test.csv`: The test dataset target labels.

4.0.3 Execution Steps

To generate the predictions and reproduce the evaluation metrics, execute the script via the command line (terminal):

```
python reproduction_script.py
```

4.0.4 Expected Output

Upon execution, the script will perform the following actions:

- Load the test data and the pre-trained pipeline.
- Generate class predictions and probability scores.
- Output the ROC AUC score, PR AUC score, Confusion Matrix, and a full Classification Report to the console.

REFERENCES

- [1]Predicting Music Track Popularity by Convolutional Neural Networks on Spotify Features and Spectrogram of Audio Waveform, Navid Falah et al
- [2]Predicting song popularity Stanford University, Stanford, CA, USA, Tech Rep, 26, 2016, James Pham, Edric Kyauk, and Edwin Park.
- [3]Groover site : <https://groover.co/en/lp/free-tools/spotify-popularity-score/>
- [4] The data set : <https://huggingface.co/datasets/maharshipandya/spotify-tracks-dataset>